

Computational Sociology

Collecting digital trace data

Dr. Thomas Davidson

University of North Carolina at Chapel Hill

September 4, 2026

Plan

- ▶ Part I: Digital trace data and ethics
 - ▶ Big data, observational data, and research ethics
 - ▶ APIs and web scraping compared
- ▶ Part II: Using APIs and web scraping in R
 - ▶ Querying the GitHub and New York Times APIs
 - ▶ Scraping a web forum using `rvest`

Part I: Digital trace data and ethics

Digital trace data

Digital traces and “big data”

“The first way that many people encounter social research in the digital age is through what is often called big data. Despite the widespread use of this term, there is no consensus about what big data even is.” - Salganik, C2.2

Digital trace data

Big data and observational data

“A first step to learning from big data is realizing that it is part of a broader category of data that has been used for social research for many years: observational data. Roughly, observational data is any data that results from observing a social system without intervening in some way.” - Salganik, C2.1

Digital trace data

Repurposing digital traces

“In the analog age, most of the data that were used for social research was created for the purpose of doing research. In the digital age, however, a huge amount of data is being created by companies and governments for purposes other than research, such as providing services, generating profit, and administering laws. Creative people, however, have realized that you can repurpose this corporate and government data for research.” - Salganik, C2.2

Digital trace data

Advantages of “big data”

- ▶ Size
 - ▶ Large-scale processes
 - ▶ Heterogeneity
 - ▶ Rare events
 - ▶ Small effects

Digital trace data

Advantages of “big data”

- ▶ Always-on
 - ▶ Longitudinal studies
 - ▶ Unexpected events

Digital trace data

Advantages of “big data”

- ▶ Non-reactive
 - ▶ Stigmatized behaviors
 - ▶ Hidden populations

Digital trace data

Disadvantages of “big data”

- ▶ Incomplete
- ▶ Inaccessible
- ▶ Non-representative
- ▶ Drift
- ▶ Algorithmic confounding
- ▶ Dirty
- ▶ Sensitive

Ethics and computational research

New ethical questions

- ▶ Salganik discusses studies that raise new ethical questions
 - ▶ Emotional contagion experiment on Facebook
 - ▶ Observational study of Facebook networks
 - ▶ Browser-based study of censorship

Ethics and computational research

Four ethical principles in social research

- ▶ *Respect for persons*
 - ▶ Treating people as autonomous and honoring their wishes
- ▶ *Beneficence*
 - ▶ Understanding risks and benefits; finding the right balance
- ▶ *Justice*
 - ▶ Even distribution of the risks and benefits
- ▶ *Respect for law and public interest*
 - ▶ Extends beyond research participants to other stakeholders
 - ▶ Compliance and transparency-based accountability

Ethics and computational research

Two ways of thinking about research ethics

- ▶ *Consequentialism*
 - ▶ Ends; consequences of research
- ▶ *Deontology*
 - ▶ Means; ethical duties, irrespective of consequences

Ethics and computational research

Four challenges in digital research

- ▶ *Informed consent*
 - ▶ When is it practical to get consent to participate?
 - ▶ When is it acceptable to proceed without consent?
- ▶ *Managing informational risk*
 - ▶ Risks of disclosure of personal information
 - ▶ Anonymization is often imperfect
- ▶ *Privacy*
 - ▶ What information is public or private?
 - ▶ Context-relative informational norms
- ▶ *Ethical decisions and uncertainty*
 - ▶ Minimal risk standard
 - ▶ Power analysis

Collecting digital trace data

Two approaches

- ▶ An *application programming interface* (API) provides data programmatically as defined by a website or platform
- ▶ *Web scraping* pulls the data directly from the web page

APIs

What is an API?

- ▶ An Application Programming Interface is a way to programmatically interact with a website
- ▶ Requests to retrieve, modify, or delete data
- ▶ APIs are typically created for developers, not academic researchers

APIs

Examples

- ▶ APIs widely used to construct observational datasets
- ▶ Many organizations have APIs including social media platforms (e.g., Facebook, TikTok), governments (e.g., Census Bureau, NYC Open Data), media (e.g., NY Times, Chronicling America), academic publishers (e.g., JSTOR, Web of Science)
- ▶ See <https://github.com/public-apis/public-apis> for a vast list of APIs
- ▶ Platforms use APIs to provide access to proprietary systems, including LLMs (e.g., Google Geocoding, Google Translate, OpenAI, Pangram)

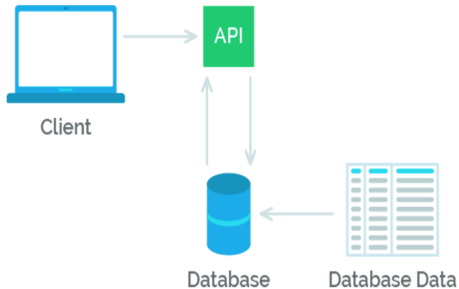
APIs

How do APIs work?

1. Construct a request
2. The client (us) sends the request to the API server
3. The server processes the request, retrieving any relevant data from a database
4. The server sends the data back to the client

How does an API work?

REST API Design



<https://sites.psu.edu/annaarsiriy/files/2019/02/Screen-Shot-2019-02-10-at-2.31.08-PM-1p26wa2.png>

Using APIs for sociological research

Optimal differentiation in the music industry

- ▶ Askin and Mauskapf (2017) use data from Spotify to measure features of music
- ▶ Theory: successful music needs to exhibit *optimal differentiation*
 - ▶ Distinct from recent offerings, but not too different to be unrecognizable

Using APIs for sociological research

Optimal differentiation in the music industry

- ▶ Using metrics from Echo Nest, a subsidiary of Spotify, they construct a measure of typicality and use it to predict chart position and duration, net of controls
 - ▶ 25,102 songs in charts 1958-2016

Musical features

Table 1. The Echo Nest Sonic Features

Attribute	Scale	Definition
Acousticness	0–1	Represents the likelihood that the song was recorded solely by acoustic means (as opposed to more electronic/electric means).
Danceability	0–1	Describes how suitable a track is for dancing. This measure includes tempo, regularity of beat, and beat strength.
Energy	0–1	A perceptual measure of intensity throughout the track. Think fast, loud, and noisy (i.e., hard rock) more than dance tracks.
Instrumentalness	0–1	The likelihood that a track is predominantly instrumental. Not necessarily the inverse of speechiness.
Key	0–11 (integers only)	The estimated, overall key of the track, from C through B. We enter key as a series of dummy variables.
Liveness	0–1	Detects the presence of a live audience during the recording. Heavily studio-produced tracks score low on this measure.
Mode	0 or 1	Whether the song is in a minor (0) or major (1) key.
Speechiness	0–1	Detects the presence of spoken word throughout the track. Sung vocals are not considered spoken word.
Tempo	Beats per minute (BPM)	The overall average tempo of a track.
Time Signature	Beats per bar/measure	Estimated, overall time signature of the track. 4/4 is the most common time signature by far and is entered as a dummy variable in our analyses.
Valence	0–1	The musical positiveness of the track.

Optimal typicality and success

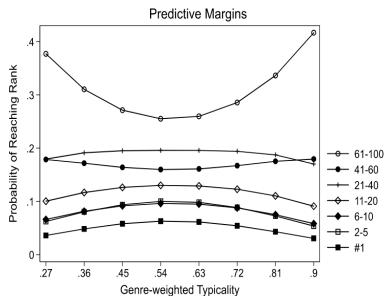


Figure 4. Predicted Marginal Probability of Songs Achieving Selected Peak Position (by Typicality) from Ordered Logit Model (Model 4)

Note: Although we inverted chart position in our models to assist readers with a more straightforward interpretation (e.g., positive coefficients reflect better performance), we revert to the originally coded chart positions for our marginal-effects graphical analysis. In the figure, the predicted positions are coded as they would be on the charts (i.e., #100 is the lowest, #1 the highest).

APIs

The post-API age?

- ▶ Access on major social media platforms
 - ▶ Limited Facebook Pages/Groups/Ads and Instagram data available and new academic initiatives in the works*
 - ▶ Twitter Academic API shut down after Musk takeover, current API too expensive for most researchers
 - ▶ Reddit had a permissive system but cut access to monetize data for large language model training
 - ▶ Restrictive conditions for TikTok academic access*

*These APIs require application processes and approval to access.

APIs

The post-API age?

- ▶ Following Facebook's decision in 2018 to close down access to its Pages API, Deen Freelon writes:
 - ▶ "We find ourselves in a situation where heavy investment in teaching and learning platform-specific methods can be rendered useless overnight."
- ▶ Recommendations
 - ▶ Learn to web scrape
 - ▶ Understand terms of service and implications of violating them

What is web scraping?

Terminology

- ▶ Web scraping is a method to collect data from websites
 - ▶ Use code underlying a web page to collect data (**scraping**)
 - ▶ Repeat process for other pages on the same website in an automated fashion (**crawling**)

What is web scraping?

Challenges

- ▶ Different websites have different structures, so a script used to scrape one will likely have to be changed to scrape another
- ▶ Websites can be internally inconsistent, making them difficult to scrape
- ▶ Some websites are easier to crawl than others, and some limit or prohibit scraping

Web scraping

Applications

- ▶ Commercial
 - ▶ Google scrapes websites to create a searchable index of the internet
 - ▶ Kayak scrapes airlines to compare flight prices
 - ▶ Recruitment companies scrape LinkedIn to get data on workers
- ▶ Social science
 - ▶ Collect data from websites without APIs, such as local newspapers, forums, small businesses, and educational institutions
 - ▶ Collect everything from a single website, e.g. all posts written on a forum
 - ▶ Collect data from many websites, e.g. all schools in a district

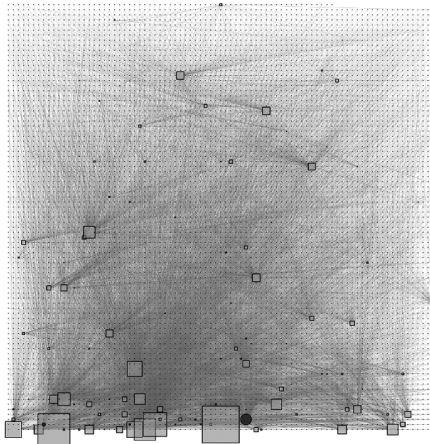
Using web scraping for sociological research

An application to the dark web

- ▶ Duxbury and Haynie (2021) collect data from an online illegal market to study how its trade structure shapes its growth
 - ▶ Use Tor browser to access Silk Road, a dark web drug market
 - ▶ Scraper runs continually to pull data on a weekly basis
 - ▶ HTML parsed to recover transactions between sellers and buyers

Using web scraping for sociological research

An application to the dark web



Using web scraping for sociological research

An application to the dark web

DESCRIPTIVE STATISTICS FOR SILK ROAD 3.1 ILLEGAL DRUG TRADE NETWORK

Measure	Mean or (%)	Range
Density	.014	
Bipartite clustering coefficient	.050	
Number of drug transactions	16,847	
Number of one-time transactions	8,690	
Number of repeat transactions	5,213	
Degree centrality (vendors)	99.69 [195.93]	0 to 1,623
Degree centrality (buyers)	2.39 [2.65]	1 to 53
Transaction costs (USD)	242.61 [585.10]	5 to 19,796
Vendor cumulative reputation	1,277.92 [1,557.54]	−5 to 8,058
Vendor negative sales ratings	−21.86 [49.73]	−406 to 0
Number of drug types sold	2.40 [1.65]	1 to 9
International shipping	21.89%	0 to 1
Vendor location:		
United States	(39.05)	0 to 1
United Kingdom	(8.28)	0 to 1
Germany	(1.78)	0 to 1
Canada	(2.34)	0 to 1
France	(5.33)	0 to 1
Australia	(.06)	0 to 1
Unknown	(43.16)	0 to 1
Number of vendors (monthly)	46.92 [30.48]	13 to 96
Number of vendors (total)	169	
Number of buyers	7,047	
Number of market actors	7,205	
Number of time periods	13	

NOTE.—Market observed from January 1, 2017, to February 1, 2018. Numbers in square brackets are SDs.

Using web scraping for sociological research

An application to the dark web

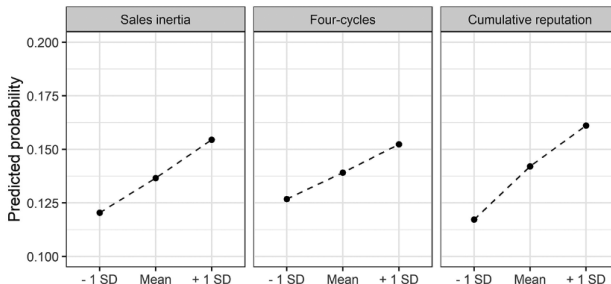


FIG. 4.—Predicted probability of illegal drug exchange given a 1-SD change in predictor variable.

Web scraping and the law

Legal considerations

No Robots, Spiders, or Scrapers: Legal and Ethical Regulation of Data Collection Methods in Social Media Terms of Service

Casey Fiesler,^{1*} Nathan Beard,² Brian C. Keegan¹

¹Department of Information Science, University of Colorado Boulder

²College of Information Studies, University of Maryland

Abstract

Researchers from many different disciplines rely on social media data as a resource. Whereas some platforms explicitly allow data collection, even facilitating it through an API, others explicitly forbid automated or manual collection processes. A current topic of debate within the social computing research community involves the ethical (or even legal) implications of collecting data in ways that violate Terms of Service (TOS). Using a sample of TOS from over one hundred social media sites from around the world, we analyze TOS language and content in order to better understand the landscape of prohibitions on this practice. Our findings show that

opportunities for digital social research, with new ways of collecting, analyzing, and visualizing data; it also allows for ordered collection, so that messy online data can become usable, well-ordered data sets (Marres and Weltevrede 2013).

However, even when data collection is possible technically, sometimes it is prohibited by terms of service (TOS), which restrict certain behaviors and uses of a site. Whether it is permissible, or ethical, for researchers to violate TOS in the course of collecting data is currently an open question within the social computing research community (Vaccaro et al. 2015; Vitak, Shilton, and Ashktorab 2016).

Web scraping and the law

Legal considerations

- ▶ Fiesler, Beard, and Keegan (2020) review the legal cases related to web scraping and analyze website terms of service
 - ▶ “In short, it is an unsettled question as to whether it is explicitly illegal (or even a criminal act) to violate TOS.”
 - ▶ No academic or journalist has ever been prosecuted for violating a website’s terms of service to collect data for research
- ▶ They analyze terms of service of over 100 social media websites
 - ▶ Terms of service are ambiguous, inconsistent, and lack context
- ▶ Recent cases have ruled in favor of legality, e.g. *hiQ Labs, Inc. v. LinkedIn Corp.*
- ▶ Advocacy for public interest web scraping

<https://themarkup.org/news/2020/12/03/why-web-scraping-is-vital-to-democracy>

Web scraping and the law

Best practices

- ▶ Only scrape publicly available data
 - ▶ i.e. You can access the page on the web without logging in
- ▶ Do not scrape copyright protected data
- ▶ Avoid violating website terms of service
- ▶ Do not burden the website
 - ▶ Limit the number of calls you make (similar to rate limits in APIs)
- ▶ Avoid using the data in a way that may interfere with business
 - ▶ e.g. Don't copy valuable data from a small business and share it on GitHub

Comparison

	API	Web scraping
Access	Granted by the platform	Visible on the web page
Format	Typically JSON	HTML
Coverage	Defined by endpoint	Anything on the page
Stability	Stable until deprecated	Breaks when the page changes or access is blocked
Usage	Documentation, keys, rate limits	robots.txt, terms of service
Effort	Low	Higher, and site-specific

Selecting the right approach

- ▶ Check for an API first
- ▶ Scrape when there is no API, when the API omits what you need, or when access has been withdrawn
- ▶ The two could be combined, using an API for part of the data and scraping for the rest

Part II: Using APIs and web scraping in R

APIs

GitHub API example

- ▶ Here is a simple call to the *users endpoint* of the GitHub API:
`https://api.github.com/users/t-davidson`
- ▶ Note the API call uses a modified version of the GitHub URL, `api.github.com`
- ▶ The result points to other endpoints. Following `followers_url` gives
`https://api.github.com/users/t-davidson/followers`
- ▶ Most APIs have documentation, explaining how each endpoint works.

APIs

Parameters and arguments

- ▶ The documentation lists other parameters we can add to a query
 - ▶ Add them as query strings using the ?
 - ▶ Each parameter has an argument specified after =
 - ▶ Separate multiple arguments using the & symbol
- ▶ What do the following queries do?
 - ▶ `https://api.github.com/users/t-davidson/followers?per_page=50`
 - ▶ `https://api.github.com/users/t-davidson/followers?per_page=100&page=2`
 - ▶ `https://api.github.com/users/t-davidson/followers?page=3`

APIs

Rate limits

- ▶ APIs use rate limits to control usage
 - ▶ How many API calls you can make
 - ▶ How much data you can retrieve
- ▶ Obey rate limits, or your credentials may be blocked
- ▶ APIs sometimes show you your rate limits,
e.g. https://api.github.com/rate_limit
 - ▶ Beware: sometimes there will be rate limits on the rate limit endpoint!

Using the GitHub API

Calling an API in R (the hard way)

```
library(httr)
library(jsonlite)
library(tidyverse)

url <- "https://api.github.com/users/t-davidson"
request <- GET(url = url)
response <- content(request, as = "text", encoding = "UTF-8")
data <- fromJSON(response)
```

See Wikipedia for a primer on UTF-8 encoding.

APIs

JSON

- ▶ An API will commonly return data in JSON (JavaScript Object Notation) format
 - ▶ JSON consists of key-value pairs, enclosed in braces as such:
`{"key": "value"}`
- ▶ It is easy to parse and converts directly into a `list` in R

New syntax

Pipes

- ▶ Pipes chain a sequence of operations together, improving readability
 - ▶ Operations read left to right rather than nesting from inner to outer
 - ▶ New syntax is `|>`, but old `%>%` also works

```
sum(sqrt(seq(1:10))) # nested
```

```
## [1] 22.46828
```

```
seq(1:10) |> sqrt() |> sum() # piped
```

```
## [1] 22.46828
```

Using the GitHub API

More efficient syntax

The three steps from the previous chunk, chained together.

```
data <- GET(url = url) |>  
  content(as = "text", encoding = "UTF-8") |>  
  fromJSON()
```

Using the GitHub API

Inspecting the results

`class` shows the object is a list. The `$` operator pulls out specific elements.

```
class(data)
```

```
## [1] "list"
```

```
data$name
```

```
## [1] "Tom Davidson"
```

```
data$followers_url
```

```
## [1] "https://api.github.com/users/t-davidson/followers"
```

Using the GitHub API

Calling another endpoint

The result of one call supplies the URL for the next. `as_tibble` converts the parsed JSON into a table.

```
followers <- GET(url = data$followers_url) |>  
  content(as = "text", encoding = "UTF-8") |>  
  fromJSON() |> as_tibble()  
  
followers |> select(login, type) |> head(3)
```

```
## # A tibble: 3 x 2  
##   login      type  
##   <chr>     <chr>  
## 1 loretoparisi User  
## 2 korymath   User  
## 3 tejastank  User
```

APIs

Access credentials

- ▶ Often APIs will use credentials to control access
 - ▶ A *key* (analogous to a user name)
 - ▶ A *secret* (analogous to a password)
 - ▶ An *access token* (grants access based on key and password)
 - ▶ Generally the access token is provided as part of the call
- ▶ Keep credentials private
 - ▶ Avoid accidentally sharing them on GitHub

Using the NYT API

Getting started

- ▶ Begin by following the instructions here to set up an account:
 - ▶ <https://developer.nytimes.com/get-started>
- ▶ We'll go through the steps for registering an app
- ▶ Add a name and a short description, e.g. "Computational Sociology", "App for class"
- ▶ Click Enable for Archive, Article Search, and Books APIs

Using the NYT API

Storing credentials

- ▶ Open `creds.json`, located in the `credentials` folder, and paste the App ID, key, and secret (click “Show secret”) into the relevant fields. Save the file.
 - ▶ Storing credentials in a separate file helps to prevent them from getting committed to GitHub accidentally

Using the NYT API

Storing credentials

- ▶ The file should look like this:

```
{"id": "1e90de-4023-8b21-493h-489fv030",  
"key": "328248djkejf298382189du329323c",  
"secret": "jw7329889d37f7798383e8d29ew2d"}
```

Note: These credentials are fake. Real ones would be leaked when these slides are shared on GitHub.

Using the NYT API

Loading packages

We're using `nytimes`, a *wrapper* around the NYT API. Rather than writing the query code ourselves, we pass query parameters to a function.

Run the lines below to install it. Follow any instructions in the Console, selecting CRAN packages if asked to update.

```
install.packages("remotes")  
remotes::install_github("t-davidson/nytimes")
```

You can read more about the library here: <https://github.com/news-r/nytimes>

Using the NYT API

Authentication

Load the packages, read in the credentials, and register the key.

```
library(nytimes)
library(tidyverse)
library(jsonlite)
library(lubridate)

creds <- read_json("../credentials/creds.json") # read creds

nytimes_key(creds$key)
```

Using the NYT API

Querying the API

```
articles <- ny_search("artificial intelligence",  
                      since = Sys.Date() - 30,  
                      pages = 15,  
                      sort = "newest")
```

Using the NYT API

Inspecting the results

The data are returned in a nested list. Indexing extracts different components. This cannot directly be transformed into tabular data.

```
head(articles[1])  
  
articles[2][[1]]$headline$main  
  
articles |> as_tibble() # Throws an error
```

Using the NYT API

Cleaning the data

This function extracts the relevant information from a single article. `lapply` applies it to each element of the list, then `bind_rows` stacks the results into a tibble.

```
extract_info <- function(x) {  
  return(tibble(  
    id           = x$`_id`,  
    headline     = x$headline$main,  
    abstract     = x$abstract,  
    pub_date     = x$pub_date,  
    section      = x$section_name,  
    url          = x$web_url,  
    word_count   = x$word_count  
  ))  
}  
  
articles_df <- lapply(articles, extract_info) |> bind_rows()
```


Using the NYT API

Creating a summary

Now that the results are tabular we can use the tidyverse verbs from R4DS. What does this query do?

```
articles_df |> group_by(section) |>  
  summarize(count = n()) |>  
  filter(count > 3) |>  
  arrange(desc(count))
```

Using the NYT API

Collecting more data

A second query, for comparison.

```
articles2 <- ny_search("Iran",  
                      since = Sys.Date() - 30,  
                      pages = 15,  
                      sort = "newest")  
  
ir <- lapply(articles2, extract_info) |> bind_rows()  
  
articles_df$topic <- "AI" # Adding a topic column  
ir$topic <- "Iran"  
both <- bind_rows(articles_df, ir)
```

Using the NYT API

Comparing the topics

```
both |> group_by(topic, section) |>
  summarize(count = n()) |>
  top_n(count, n=4) |>
  ggplot(aes(y=count, x = section, fill = section)) +
  geom_bar(stat = "identity") +
  scale_fill_viridis_d(option = "cividis") +
  facet_wrap(~topic, scales = "free_x") +
  theme_bw() +
  labs(y = "Number of articles", x = "Section",
       title = "Top 4 sections by topic") +
  theme(legend.position = "none")
```

Using the NYT API

Comparing coverage over time

```
both |> mutate(day = as.Date(pub_date)) |>
  group_by(topic, day) |>
  summarize(coverage = sum(word_count)) |>
  ggplot(aes(y = coverage, x = day, color = topic)) +
  geom_point() + geom_line() + theme_bw() +
  labs(y = "Total words", x = NULL, color = "Topic",
       title = "AI and Iran coverage
       in the NYT")
```

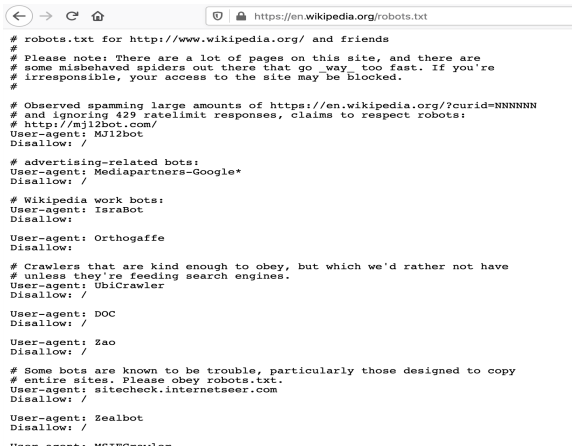
APIs

Best practices

- ▶ Use a wrapper package if available
 - ▶ Although sometimes you will have to write your own queries
- ▶ Build functions to obey rate limits where possible
- ▶ Access only the data you need
- ▶ Test using small examples before collecting a larger dataset

How to scrape a web page

Start by looking up robots.txt

A screenshot of a web browser window. The address bar shows the URL "https://en.wikipedia.org/robots.txt". The page content is a text file with various directives for web crawlers. The text is as follows:

```
# robots.txt for http://www.wikipedia.org/ and friends
#
# Please note: There are a lot of pages on this site, and there are
# some misbehaved spiders out there that go way too fast. If you're
# irresponsible, your access to the site may be Blocked.
#
# Observed spamming large amounts of https://en.wikipedia.org/?curid=NNNNNN
# and ignoring 429 ratelimit responses, claims to respect robots:
# http://mjl2bot.com/
User-agent: MJl2bot
Disallow: /

# advertising-related bots:
User-agent: Mediapartners-Google*
Disallow: /

# Wikipedia work bots:
User-agent: IsraBot
Disallow:

User-agent: Orthogaffe
Disallow:

# Crawlers that are kind enough to obey, but which we'd rather not have
# unless they're feeding search engines.
User-agent: UbiCrawler
Disallow: /

User-agent: DOC
Disallow: /

User-agent: Zao
Disallow: /

# Some bots are known to be trouble, particularly those designed to copy
# entire sites. Please obey robots.txt.
User-agent: sitecheck.internetseer.com
Disallow: /

User-agent: Zealbot
Disallow: /

User-agent: MCFBot
```

How to scrape a web page

Decoding robots.txt

- ▶ **User-agent** = the name of the scraper
 - ▶ * = All scrapers
- ▶ **Allow**: /path/ = OK to scrape
- ▶ **Disallow**: /path/ = Not OK to scrape
 - ▶ **Disallow**: / = Not OK to scrape any pages
- ▶ **Crawl-delay**: N = Wait N seconds between each call to the website

How to scrape a web page

Exercise

- ▶ Find a website of interest
- ▶ Locate the robots.txt file
 - ▶ Does the website allow web scraping?
 - ▶ Are there any restrictions on which pages can be accessed?

How to scrape a web page

Terminology

- ▶ A web page is loaded using a **URL** (Uniform Resource Locator)
- ▶ Open a website and right click on any text or image on the screen
 - ▶ You should see an option **Inspect Element**, which shows the code used to generate the page
- ▶ It is usually displayed by rendering **HTML** (Hypertext Markup Language)
- ▶ Many websites use **CSS** (Cascading Style Sheets) to style the page, which can help for navigation

See tutorial on CSS <https://flukeout.github.io/> and Chrome Extension <https://selectorgadgets.com/>

How to scrape a web page

Using rvest to scrape HTML

Today's example is Hacker News, a news and discussion forum. We start with a single thread, a discussion about a *New Yorker* article on AI and loneliness.

```
library(rvest)
library(tidyverse)
library(lubridate)

base <- "https://news.ycombinator.com/"
url <- paste0(base, "item?id=44766508")
thread <- read_html(url)

class(thread)
```

How to scrape a web page

Is scraping allowed?

```
User-Agent: *  
Crawl-delay: 30  
Disallow: /login  
Disallow: /vote?  
Disallow: /reply?
```

<https://news.ycombinator.com/robots.txt>

How to scrape a web page

Collecting comments

Each comment's text carries the class `commtext`.

```
comments <- thread |> html_nodes(".commtext") |>
  html_text() |> str_trim()

length(comments) # 1034
substr(comments[1], 1, 200)
```

How to scrape a web page

Different selectors give different results

Comments sit in table rows of class `comtr`. In theory, counting the rows, the names, and the texts should give the same number three times. Why are they different?

```
length(thread |> html_nodes("tr.comtr"))    # 1037
length(thread |> html_nodes(".hnuser"))      # 1038
length(thread |> html_nodes(".commtext"))    # 1034
```

How to scrape a web page

Select the rows first, then the fields

Comment text and user are a subset of comment objects. Collecting these provides the relevant contents.

```
rows <- thread |> html_nodes("tr.comtr")

length(rows |> html_node(".hnuser"))    # 1037
length(rows |> html_node(".commtext"))  # 1037
```

How to scrape a web page

Timestamps and reply depth

Some useful values are stored in attributes rather than text so are not directly shown on the page. `html_attr` can retrieve this information.

```
rows |> html_node(".age") |> html_attr("title") |> ymd_hms() |>  
  head(3)
```

```
rows |> html_node("td.ind") |> html_attr("indent") |>  
  head(3)
```

How to scrape a web page

Wrapping it in a function

Every field is taken from rows, using `html_node` throughout, so the columns cannot drift out of alignment.

```
get.comments <- function(thread) {  
  rows <- thread |> html_nodes("tr.comtr")  
  data <- tibble(  
    user = rows |> html_node(".hnuser") |> html_text(),  
    time = rows |> html_node(".age") |>  
      html_attr("title") |> ymd_hms(),  
    depth = rows |> html_node("td.ind") |>  
      html_attr("indent") |> as.integer(),  
    text = rows |> html_node(".commtext") |>  
      html_text() |> str_trim()  
  )  
  return(data)  
}
```


How to scrape a web page

Using the function

```
d <- get.comments(thread)

dim(d)
head(d |> select(user, time, depth))

table(d$depth) # how deep does the conversation go?
```

How to scrape a web page

Crawling a website

One thread is rarely enough.

```
f <- read_html(paste0(base, "news"))  
  
rows <- f |> html_nodes("tr.athing.submission")
```

How to scrape a web page

Two more selectors that mislead

```
length(rows)
length(f |> html_nodes(".titleline a"))
length(f |> html_nodes(".score"))
```

- ▶ Each row holds a link to the article *and* a link to its domain, so `.titleline a` finds both. `.titleline > a:first-child` takes only the first
- ▶ Job postings carry no score, so `.score` can return fewer values than there are rows, and every column after it is shifted

How to scrape a web page

Collecting the stories

```
get.stories <- function(f) {  
  rows <- f |> html_nodes("tr.athing.submission")  
  sub <- f |> html_nodes("td.subtext")  
  data <- tibble(  
    title = rows |>  
      html_node(".titleline > a:first-child") |> html_text(),  
    id = rows |> html_attr("id"),  
    user = sub |> html_node(".hnuser") |> html_text(),  
    score = sub |> html_node(".score") |>  
      html_text() |> parse_number(),  
    time = sub |> html_node(".age") |>  
      html_attr("title") |> ymd_hms()  
  )  
  return(data)  
}  
stories <- get.stories(f)
```

How to scrape a web page

Pagination

The link to the next page of stories has the class `morelink`. With `html_node` again, a page with no such link returns `NA` rather than an empty vector.

```
get.next.page <- function(f) {  
  nxt <- f |> html_node(".morelink") |> html_attr("href")  
  if (is.na(nxt)) {  
    return("Final page")  
  }  
  return(paste0(base, nxt))  
}  
  
get.next.page(f)
```

How to scrape a web page

Paginate and scrape

`Sys.sleep(30)` honors the crawl delay, so the loop spends most of its life waiting. That is why scrapers are left to run unattended.

```
paginate.and.scrape <- function(url, max.pages = 3) {  
  f <- read_html(url)  
  stories <- get.stories(f)  
  next.page <- get.next.page(f)  
  p <- 1  
  
  while (next.page != "Final page" & p < max.pages) {  
    Sys.sleep(30)  
    f <- read_html(next.page)  
    stories <- bind_rows(stories, get.stories(f))  
    next.page <- get.next.page(f)  
    p <- p + 1  
  }  
  return(stories)  
}
```

How to scrape a web page

Paginate and scrape

```
front <- paginate.and.scrape(paste0(base, "news"))  
  
dim(front)  
front |> arrange(desc(score)) |> select(title, score) |>  
  head(5)
```

Each story id is the address of its discussion, so `get.comments` can now be run over `front$id` to collect the comments as well. At thirty seconds a page, that is a job to leave running.

How to scrape a web page

Storing the results

```
write_csv(front, "../data/hn_front.csv")
```


How to scrape a web page

Storing the results

- ▶ If you collect everything before saving it, you risk losing it all when the script crashes
- ▶ Solutions
 - ▶ Continuously save results to disk, or store them in chunks
 - ▶ More robustly, write output to a relational database such as PostgreSQL
- ▶ Log progress: print statements and a log file
- ▶ If collecting a lot of data, run it on a server

Advanced web scraping

Javascript and browser automation

- ▶ Many websites build their content with Javascript and do not provide an HTML version
- ▶ We can get around this by automating a browser
 - ▶ Open a browser, load the website, read the HTML from the browser into R
- ▶ Selenium WebDriver, via the package `RSelenium`, is the most popular approach
 - ▶ `RSelenium` requires a complicated setup using a Docker container; it is easier to use `selenium` in Python, then read the results into R using `reticulate`
- ▶ Browser automation can also click buttons, fill forms, log in, and save images

Concluding remarks

- ▶ Both APIs and web scraping can provide high-quality digital trace data
- ▶ Changing access and structures add difficulty
- ▶ Ethical considerations are important, regardless of the data source

Next week

- ▶ Online surveys and experiments